Guided Lab 01: Write and Run Your First Playbook (Nginx Install)

Lab Objective

By the end of this lab, you will:

- Create a clean project folder for your Ansible work
- Write a simple playbook to install `nginx` on a remote Ubuntu server
- Run the playbook using `ansible-playbook`
- Learn how to interpret Ansible output
- Understand and test **idempotence**
- Verify that `nginx` is running — using both CLI and browser

Lab Prerequisites

Before you begin, make sure:

- You have **Ansible installed** on your **control node** (your local machine or a cloud VM like Azure)
- You can **SSH into your managed node** (Ubuntu VM) using a private key
- Your managed VM has:
  - Public IP address
  - Port 22 (SSH) open
  - Python installed (comes by default on most Ubuntu images)

If you've completed the labs in earlier sections, you're all set.

Step 1: Create a Project Folder

Start by organizing your work. Open your terminal and create a dedicated folder for this lab.

```
mkdir nginx-setup
cd nginx-setup
```

This will help you keep your inventory, playbook, and supporting files all in one place.

Step 2: Create Your Inventory File

Let's define the servers Ansible will connect to.

Create a file called `inventory.ini` :

```
nano inventory.ini
```

Paste the following (replace the IP with your VM's public IP):

```
[web]
<YOUR_VM_PUBLIC_IP>

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Explanation:

- `[web]` : a group of web servers (we're targeting this in our playbook)
- `ansible_user` : your VM's SSH username (use `azureuser` for Azure)
- `ansible_ssh_private_key_file` : the path to your SSH private key

Step 3: Test SSH Connection with Ansible

Before writing the playbook, make sure Ansible can talk to your VM.

Run:

```
ansible all -i inventory.ini -m ping
```

Expected output:

```
<YOUR_VM_PUBLIC_IP> | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

If this fails:

- Check your IP and username
- Ensure port 22 is open in your VM's network security group
- Make sure your private key path is correct

Step 4: Create Your First Playbook File

Now let's write the playbook that installs nginx.

Create a file named `install-nginx.yml` :

```
nano install-nginx.yml
```

Paste the following YAML:

```yaml
- name: Install nginx on Ubuntu server
  hosts: web
  become: yes
  remote_user: azureuser

  tasks:
    - name: Ensure nginx is installed
      apt:
        name: nginx
        state: present
```

What this playbook does:

- Connects to the `web` group
- Escalates privileges with `become: yes` to run `apt`
- Installs `nginx` using the `apt` module
- Ensures the package is only installed if not already present (idempotent)

Step 5: Run the Playbook

Use the `ansible-playbook` command to execute your playbook:

```
ansible-playbook -i inventory.ini install-nginx.yml
```

You should see:

```
PLAY [Install nginx on Ubuntu server] ***************************

TASK [Gathering Facts] ******************************************
ok: [<your-vm-ip>]

TASK [Ensure nginx is installed] ********************************
changed: [<your-vm-ip>]
```

If the task shows `changed`, Ansible installed nginx. If it says `ok`, nginx was already installed.

Step 6: Test Idempotence

Now re-run the same playbook:

```
ansible-playbook -i inventory.ini install-nginx.yml
```

This time, you should see:

```
ok: [<your-vm-ip>]
```

That means Ansible didn't repeat the installation — because the desired state was already achieved.

This is called **idempotence**, and it's a key principle in automation.

Step 7: Verify nginx Is Running

Option 1: Use `curl` from your control node

```
curl http://<your-vm-public-ip>
```

You should see HTML output from the Nginx default page.

Option 2: Open in your browser

Go to:

```
http://<your-vm-public-ip>
```

You should see the Nginx welcome page.

If not:

- Check that port 80 is open in the VM's firewall settings
- Restart nginx manually with:

```
sudo systemctl restart nginx
```

Step 8: Use Verbose Mode (Optional)

To see more detail about what Ansible is doing behind the scenes, use:

```
ansible-playbook -i inventory.ini install-nginx.yml -v     # minimal
ansible-playbook -i inventory.ini install-nginx.yml -vv    # medium
ansible-playbook -i inventory.ini install-nginx.yml -vvv   # detailed
```

Use `-vvv` when troubleshooting or when you want to understand exactly what's happening.

Assignment (Optional but Recommended)

Create a second playbook `uninstall-nginx.yml` that:

- Stops the nginx service
- Removes the nginx package

Use modules like `service` and `apt` with `state: absent`.

Then re-run your original `install-nginx.yml` to verify nginx comes back cleanly.

This helps reinforce:

- Writing multiple playbooks
- Understanding playbook flow
- Working with service and apt modules together

Lab Summary

In this lab, you:

- Created a project folder and inventory file
- Wrote your first playbook in YAML

- Installed nginx on a remote Ubuntu server using Ansible
- Verified automation using browser and curl
- Learned about idempotence, verbosity, and modularity